INTER COMPANY SELL MODULE

Technical & Functional Documentation

GROO Retail ERP (Woven Essence) · Next.js 15 + MongoDB

Screens: Consignment (Delivery Challan) · Receive Delivery Challan · Sales Invoice · Auto Purchases Receiveds · Return Approval

Prepared: 21-10-2026

Prepared by: _____ Reviewed by: _____

Contents

1. Overview & sidebar
2. The business flow in one page
3. Architecture & shared building blocks
4. Screen: Consignment / Delivery Challan
5. Screen: Receive Delivery Challan
6. Screen: Sales Invoice (+ E-Invoice)
7. Screen: Auto Purchases Receiveds
8. Screen: Return Approval
9. End-to-end flow: what changes where
10. Data models
11. API reference
12. Roles & Permissions
13. Library helpers
14. Configuration (Doc Setup, E-Invoice)
15. Recent changes (October 2026)
16. How to run, test and debug

All paths are relative to the project root `wovenessenceV2/`. Line numbers are approximate (21-10-2026). Several files keep older commented-out versions above the live code (e.g. `IcChallanForm.jsx` live code starts ~line 519).

1. Overview & sidebar

Inter Company Sell moves goods **between branches/companies of the same group** on consignment: the sender ships stock on a Delivery Challan, the receiver approves it into its own stock, sells it at its POS, and the sender then bills only what was sold (Sales Invoice, with GST e-invoice). The receiver records that invoice as a purchase (Auto Purchases Receiveds). Damaged goods can be returned through a request / approval handshake.

Sidebar label	Route	Who uses it
Consignment (Delivery Challan)	`/admin/transaction/intercompanysell/deliverychallan`	Sender
Receive Delivery Challan	`/admin/transaction/intercompanysell/receivedeliverychallan`	Receiver
Sales Invoice	`/admin/transaction/intercompanysell/sales-invoice`	Sender
Auto Purchases Receiveds	`/admin/transaction/intercompanysell/auto-purchase-receiveds`	Receiver
Return Approval	`/admin/transaction/intercompanysell/consignment`	Sender (approves), receiver (sees status)

Sidebar definition: `config/nav.js` (~lines 504-534). "Sender" / "receiver" always means the branch selected in the top bar.

2. The business flow in one page

#	Step	Screen	Who	Effect
1	Ship goods on a Delivery Challan	Consignment → Add	Sender	Sender's stock leaves (IN_TRANSIT); challan is "Pending".
2	Approve the challan	Receive Delivery Challan	Receiver	New stock rows created under the receiver (IN_STOCK); challan "Accepted".
3	Sell at the counter	Sell → POS	Receiver	Unit sold; a `sales[]` entry marks it billable to the sender.
4	Bill what was sold	Sales Invoice	Sender	One invoice per receiver; sold units become "Billed".
5	Generate GST e-invoice (IRN)	Sales Invoice → E-Invoice	Sender	IRN / Ack / QR stored on the invoice.
6	Record the purchase	Auto Purchases Receiveds	Receiver	Purchase record with product group & HSN per line (needs the IRN first).
7	Request a return (damaged)	Receive DC → View popup	Receiver	Pending return request; nothing moves.
8	Approve / reject the return	Return Approval	Sender	Approve: stock goes back to the sender. Reject: nothing moves.

3. Architecture & shared building blocks

- Pages under `app/admin/transaction/intercompanysell/**`; API routes under `app/api/ic-*`.

- **Stock** is the `barcodeLabel` collection (one row per unit/batch). IC stock maths is in `lib/icStock.js` and `lib/icReceive.js`.
- **Consignment list** uses the generic `components/ListView.jsx` (numbered pager + header sorting: `cfg.pager:'numbered'`, `cfg.sortable:true`; server side `lib/listSort.js sortedPage`).
- **The other four screens are custom pages** that load their whole list in one request; their tables use `useClientTable` from `components/NumberedPager.jsx` (sorts and pages the rows already loaded: `{rows, offset, th(), pager, firstPage()}`).
- **Permissions**: `lib/screenPermission.js` (SCREENS.IC_*), see section 12.

4. Screen: Consignment / Delivery Challan

Route	File	Renders
List	`deliverychallan/page.jsx`	ListView twice: **Pending Delivery Challans** (`received:'no'` , slotted in as `beforeTable`) and **Inter Company Delivery Challans** (`received:'yes'` , with summary cards Total DC No / Total Qty and filters To Business, Start/End Date). Status badge: Accepted (green) / Pending (yellow) from `receivedAt` .
Add / Edit	`deliverychallan/add/page.jsx` , `[id]/page.jsx`	`components/IcChallanForm.jsx` (compact grid; after save goes to the print page)
Print	`deliverychallan/print/[id]/page.jsx`	`components/IcChallanPrintView.jsx`
Field spec	`deliverychallan/fields.js`	FIELDS (~286), COMPACT_GRID_COLS (~339), **`computeLine`** (~365), **`computeTotals`** (~383) - shared by browser and API

IcChallanForm (Add / Edit)

- Destination company: `GET /api/ic-destinations?business=` (every other business; auto-selected if only one). Destination location: `GET /api/options?ref=companylocations&business=<to>` (first auto-selected). GSTIN / address copied from `GET /api/business/<id>` .
- Stock point auto-picked (`useOptions('stockpoint')`).
- **Scan** (`addScanned` ~764): `GET /api/ic-delivery-challan/item-lookup?code&business&location&finYear&toBusiness&stockPoint` ; one line per barcode; qty defaults to the available quantity. Type-ahead: `GET /api/ic-delivery-challan/item-codes` (250 ms debounce).
- Line maths (`computeLine`): Discount = Rate × %; Final = Rate - Discount - R.Off; Before Tax = Final × Qty; each GST = Before Tax × slab %; Net = sum. Totals: header discount, net rounded to the rupee, round off = difference.
- Submit (~862): `POST /api/ic-delivery-challan` (new) or `PUT /api/ic-delivery-challan/<id>` (edit) → print page. Close button → list (21-10-2026).

IcChallanPrintView

Loads the challan and both businesses (`GET /api/business/<from>` , `<to>`) for the letterheads; quantity totals grouped PC/MTR (`uomTypeOf`); the challan shows **no money**. Buttons: Print, Back.

API

Call	What it does
`GET /api/ic-delivery-challan` app/api/ic-delivery-challan/route.js ~99	IC_DELIVERY_CHALLAN read. Filters business, location, finYear, toBusinessId, toLocationId, dates, `received=yes|no` , search (dcNo, GSTN, address). Sorting via `sortedPage` (keys dcNo, dcDate, totalQty, createdAt, netValue, receivedAt; refs toBusinessId, toLocationId, stockPointId). Summary counts ignore the received split. Rows get a joined `barcodeNo` string.
`POST /api/ic-delivery-challan` (~240)	Create: validate FIELDS + at least one item → `checkRoute` (not to itself) → `applyTotals` recomputes on the server → number **`DC/<yy-yy>/<branch initials>/<999+seq>`** via Counter `icDcNo|<branch>` (not Doc Setup) → create → **`shipChallanStock`** (stock leaves the sender, writes `items[].stockMoves`); on failure the challan is deleted and 422 returned.

`GET/PUT/DELETE /api/ic-delivery-challan/[id]`	Read / update / delete on the challan's own business. PUT and DELETE are refused (409) if an old-flow invoice is linked (`icSalesInvoiceId`).
`GET /api/ic-delivery-challan/item-lookup`	Finds an **IN_STOCK** unit by barcodeNo / barcodeGenerated / oldBarcode in this business; clear errors for item-code typed, not found, not in stock, other branch. Enriches Item → HSN → tax slab (fallback: unit's GST split). **IGST vs CGST+SGST**: compares the first two digits (state code) of sender and receiver GSTIN. Rate = Item RSP (fallback unit retail price). maxQty = sum of IN_STOCK qty.
`GET /api/ic-delivery-challan/item-codes`	Up to 25 IN_STOCK barcodes for type-ahead.
`GET /api/ic-destinations`	Every other business (`lib/icRouting.allowedDestinations`).

5. Screen: Receive Delivery Challan

Page: `receivedeliverychallan/page.jsx` (custom). Two loads of `GET /api/ic-receive-delivery-challan?view=incoming&perPage=100` : `received=no` (**Pending** section, highlighted, with Approve) and `received=yes` (**Received** section).

- **Approve**: `POST /api/ic-receive-delivery-challan {id, business}` (no action = receive).
- **View popup**: lines, per-line **Return Qty** (capped by `leftToReturn` = min(qty - returned, still on hand) - already requested), return history and request statuses. **Request Return**: `POST {id, business, action:'return', lines}` .
- Both tables: sortable headers + pager (`useClientTable`).

API: `app/api/ic-receive-delivery-challan/route.js`

Call	What it does
GET `view=incoming`	IC_RECEIVE_DELIVERY_CHALLAN read. Challans addressed to this business (optionally this location), split by `receivedAt` . Received rows get `stockLeft` (`receivedInStock()`).
GET `view=returns`	IC_RETURN_APPROVAL read. Challans with returns or return requests where this business is sender or receiver.
POST (receive)	Addressee only. `lib/icReceive.receiveChallan()` : creates the receiver's stock rows, stamps `receivedAt/receivedBy` . 409 if already received.
POST `action:'return'` (`requestReturn` ~273)	Receiver only, after receipt. Qty ≤ on-hand allowance. Pushes a **pending** `returnRequests` entry (with `rid`). **No stock moves.**
POST `approve-return` / `reject-return`	IC_RETURN_APPROVAL update; sender only (section 8).

6. Screen: Sales Invoice (+ E-Invoice)

Page: `sales-invoice/page.jsx` (custom). Loads `GET /api/ic-item-invoice?view=notbilled` and `?view=billed`.

- **Receiver filter**: Company (from `/api/ic-destinations`) then Location (first auto-selected). Changing either clears the ticks.
- **Not Billed**: one row per sold unit at a receiver; tick rows (header box ticks all visible rows across pages).
- **Selected Items** preview: grouped per receiver, totalled exactly as the server will (paise-rounded sums, net rounded to the rupee, round off = difference; CGST+SGST or IGST with slab label).
- **Save**: `POST /api/ic-item-invoice {business, location, finYear, data:{barcodeIds}}` → one invoice **per receiver**.
- **Billed** table actions: View (popup), Print (new tab), **E-Invoice** button - yellow = no IRN (opens Generate popup), green = generated (opens the e-invoice print).
- Tables use `useClientTable`.

Print

`sales-invoice/print/[id]/page.jsx` → `components/IcTaxInvoiceView.jsx` with `/api/ic-item-invoice/<id>/print` (+ `?einv=1` for the e-invoice copy). Shows HSN summary, totals, amount in words; in e-invoice mode a MOCK / CANCELLED banner, IRN, Ack and QR, and the `EinvoicePanel` (preview, generate, cancel). Back button.

API

Call	What it does
`GET /api/ic-item-invoice` (~177)	IC_ITEM_INVOICE read. `billed`: sender's last 100 invoices (einvoice reduced to status/mode/irn/ack). `notbilled`: this sender's challans → receiver barcode rows with `icChallanId` in them that have a sale not yet billed (max 500) → `billable()` → priced by `buildLine()`.
Pricing (`buildLine`)	Finds the challan line whose `stockMoves` contain the row's `sourceBarcodeId` and runs `computeLine` with the sold qty - the sender bills at the **challan rate**, not the POS price. Qty = sold - returned.
`POST /api/ic-item-invoice` (~274)	Create. Validates every key (row exists, came from this sender's challan, still billable; else 409); **groups by receiver**; number `nextDocNumber(IcItemInvoice,'invoiceNo','Inter Company Sales Invoice')`; totals; **claims** each unit (stamps `sales.$.icBilledId/icBilledNo` or the row-level fields). If a claim count mismatches, that receiver's invoice is undone and 409 returned. 201 {invoices[]}.
`GET /api/ic-item-invoice/[id]/print`	Seller (Business + Location) and buyer blocks; with `?einv=1` and a GENERATED IRN adds IRN, Ack, QR (data URL from `signedQrCode`).
`GET /api/ic-item-invoice/[id]/einvoice`	Mode, current e-invoice state, the NIC payload preview and a list of `problems`.
`POST .../einvoice {action:'generate'}`	IC_ITEM_INVOICE **create**. Claims the invoice (PENDING), calls `generateIrn`; success → GENERATED (IRN, Ack, signed QR/invoice); failure → FAILED (502). Payload problems block sandbox/live (422) but not mock.
`POST .../einvoice {action:'cancel'}`	IC_ITEM_INVOICE **delete**. Only GENERATED, within 24 h of Ack date, same mode; reason 1-4. Sets CANCELLED; the invoice and billed units stay.

7. Screen: Auto Purchases Receiveds (receiver)

Page: `auto-purchase-receiveds/page.jsx` (custom). Loads `GET /api/ic-purchase-received?view=pending|received` and the pickers `/api/options?ref=product/group` and `?ref=hsn`.

- **Pending**: one card per invoice addressed to this branch **that has a GENERATED IRN**. Each line has Product Group (pre-selected from the API's `suggestedGroup`) and HSN (defaults to the invoice HSN) pickers; edits survive refresh. **Save**: `POST /api/ic-purchase-received {data:{invoiceId, lines}}`.
- **Received**: search + sortable table + View popup showing Old (as invoiced) vs Received (group, HSN; * where HSN changed).

Call	What it does
`GET /api/ic-purchase-received`	IC_PURCHASE_RECEIVED read. received: last 200 records. pending: invoices to this business with `einvoice.status='GENERATED'` not yet received (max 200); lines get `suggestedGroup` (barcode row group → item sub-group → product group name).
`POST /api/ic-purchase-received`	Create. Invoice must be to this business, IRN generated, not yet received. Number = `nextDocNumber(...,'Auto Purchase Received')` (prefixed "APR" if it starts with a digit). Unique per invoice (duplicate → 409). **Record only - no stock movement** (stock already arrived at step 2).

8. Screen: Return Approval (sender)

Page: `consignment/page.jsx` (custom). Loads `GET /api/ic-receive-delivery-challan?view=returns&received=no&perPage=100`.

- **Pending Return Requests**: every pending `returnRequests` entry. The sender sees **Approve / Reject** (`POST {id, business, action:'approve-return'|'reject-return', rid}`); without Update permission it shows "No permission to approve"; the requesting branch sees "Awaiting sender approval".
- **Returns**: business, DC, returned barcodes, totals, Returned (qty, or Awaiting approval / Rejected); read-only View popup.

Action	Server effect (`app/api/ic-receive-delivery-challan/route.js`)
Approve (`approveReturn` ~342)	Sender only, request pending; re-checks on-hand (receiver may have sold since). One guarded update: `items[].returnedQty` up, request → approved, entry pushed to `returns[]`. Then `withdrawReceivedStock` (receiver rows lose qty; emptied rows → **VOID**) and `restoreReturnedStock` (sender rows back to **IN_STOCK**, StockMovement TRANSFER_RETURN_IN). StockAdjustment ISSUE under the receiver ("Inter Company Return - Damaged").
Reject (`rejectReturn` ~437)	Request → rejected. Nothing moves.

9. End-to-end flow: what changes where

#	Step	Collections written
1	Raise challan (sender)	**icdeliverychallan** created (`receivedAt=null` , `items[].stockMoves`). **barcodeLabel** (sender rows): qtyNum reduced; emptied rows → IN_TRANSIT. **StockMovement** TRANSFER_OUT.
2	Approve (receiver)	`receiveChallanStock` : **new barcodeLabel rows** under the receiver (IN_STOCK, with `sourceBarcodeId` , `icChallanId` , `icChallanNo` ; idempotent). **StockMovement** TRANSFER_IN. **StockAdjustment** RECEIPT. Challan `receivedAt/receivedBy` stamped.
3	POS sale (receiver)	`lib/inventory.js` sell: qtyNum reduced (SOLD at 0) and a `sales[]` entry pushed with `icBilledId:null` . A POS return increments `sales.returnedQty` .
4	Sales Invoice (sender)	**iciteminvoice** created per receiver; the receiver's sale entries stamped `icBilledId/icBilledNo` (Not Billed → Billed).
5	E-Invoice (sender)	`iciteminvoice.einvoice` : PENDING → GENERATED / FAILED; CANCELLED within 24 h.
6	Auto Purchase Received (receiver)	**icpurchasereceived** created (needs GENERATED IRN). No stock change.
7	Return request (receiver)	Challan `returnRequests[]` pending entry. Nothing moves.
8a	Approve return (sender)	Challan returnedQty / returns[]; receiver rows reduced (VOID when empty); sender rows IN_STOCK + TRANSFER_RETURN_IN; StockAdjustment ISSUE.
8b	Reject return	Request → rejected.

> **GST:** IGST vs CGST/SGST is decided once, when the line is scanned on the challan (state codes of the two GSTINs), stored on the challan line and carried into the invoice. The e-invoice uses `SupTyp:'B2B'` , `IgstOnIntra:'N'` .

10. Data models

Model	Collection	Key fields
`models/IcDeliveryChallan.js`	icdeliverychallan	businessId, locationId, finYear (sender); toBusinessId, toLocationId; dcNo, dcDate, stockPointId; totals (taxableValue, igst/cgst/sgstTotal, roundOff, totalQty, netValue); **receivedAt / receivedBy** (null = pending); **returns[]** {at, by, approvedBy, lines}; **returnRequests[]** {rid, at, by, lines, status pending/approved/rejected, actedAt, actedBy}; icSalesInvoiceId (old flow); **items[]** (barcodeNo, itemCode, itemName, hsn, uom, qty, unitRate, discountPct, roffDiscount, finalRate, beforeTax, igst/cgst/sgst Pct & Amount, netAmount, remarks, **stockMoves[]** {barcodeId, qty}, returnedQty)
`models/IcItemInvoice.js`	iciteminvoice	sender scope; toBusinessId, toLocationId, toBusinessName; invoiceNo, invoiceDate; items[] (one per sold unit: barcodeId, sourceBarcodeId, barcodeNo, dcId, dcNo, qty, pricing, tax, posInvoiceNo, soldAt); totals; **einvoice** {status PENDING/GENERATED/CANCELLED/FAILED, mode mock/sandbox/live, irn, ackNo, ackDate, signedQrCode, signedInvoice, error, cancel fields, previous}
`models/IcPurchaseReceived.js`	icpurchasereceived	receiver scope; recNo, receivedAt; icItemInvoiceId (**unique**), invoiceNo, irn; from business/location; items[] with productGroupId/Name, hsn, invoiceHsn; totals
`lib/barcodeLabel.js`	barcodeLabel	Statuses IN_STOCK, IN_TRANSIT, SOLD, RETURN_IN_TRANSIT, VOID. IC fields: sourceBarcodeId, icChallanId, icChallanNo, icBilledId/No, **sales[]** {invoiceId, invoiceNo, qty, at, returnedQty, icBilledId, icBilledNo}
Others	-	StockMovement (TRANSFER_OUT / IN / RETURN_IN), StockAdjustment (RECEIPT / ISSUE), Counter (sequences), DocSetup, Business, CompanyLocation, Item, Hsn, Tax, ProductGroup, PosInvoice

11. API reference

Route	Methods	Permission	Purpose
`/api/ic-delivery-challan`	GET, POST	IC_DELIVERY_CHALLAN read / create	List; raise challan (ships stock)
`/api/ic-delivery-challan/[id]`	GET, PUT, DELETE	read / update / delete	One challan
`/api/ic-delivery-challan/item-lookup`, `/item-codes`	GET	IC_DELIVERY_CHALLAN read, create or update	Scan / type-ahead
`/api/ic-destinations`	GET	IC_DELIVERY_CHALLAN or IC_ITEM_INVOICE (read/create/update)	Destination companies

`/api/ic-receive-delivery-challan`	GET, POST	incoming/receive/return: IC_RECEIVE_DELIVERY_CHALLAN read/update; returns/approve/reject: IC_RETURN_APPROVAL read/update	Receive, return request, approve/reject
`/api/ic-item-invoice`	GET, POST	IC_ITEM_INVOICE read / create	Not billed / billed; raise invoices
`/api/ic-item-invoice/[id]/print`	GET	IC_ITEM_INVOICE read	Invoice print data
`/api/ic-item-invoice/[id]/einvoice`	GET, POST	read / generate = create / cancel = delete	E-invoice
`/api/ic-purchase-received`	GET, POST	IC_PURCHASE_RECEIVED read / create	Auto purchase received
`/api/options`, `/api/business/[id]`	GET	- / BUSINESS_HEADER_SCREENS	Pickers / letterheads

12. Roles & Permissions

Constant (`lib/screenPermission.js`)	href
IC_DELIVERY_CHALLAN	/admin/transaction/intercompanysell/deliverychallan
IC_RECEIVE_DELIVERY_CHALLAN	/admin/transaction/intercompanysell/receivedeliverychallan
IC_ITEM_INVOICE	/admin/transaction/intercompanysell/sales-invoice
IC_PURCHASE_RECEIVED	/admin/transaction/intercompanysell/auto-purchase-receiveds
IC_RETURN_APPROVAL	/admin/transaction/intercompanysell/consignment

- Rules: Super Admin unrestricted; a role with no saved matrix anywhere is allowed; once a role has a matrix, unticked screens are denied (403). A role governed elsewhere but not in this business is denied (SCREEN_FORBIDDEN_SCOPE).
- Lookups used while creating/editing (item-lookup, item-codes, destinations) accept Read **or** Create **or** Update so a create-only role can still raise a challan.
- Client side: Return Approval checks `scope.can('/admin/transaction/intercompanysell/consignment','update')` to show "No permission to approve". The API check is the real enforcement.
- `BUSINESS_HEADER_SCREENS` includes the DC and RDC screens so `GET /api/business/[id]` works for both letterheads.

13. Library helpers

File	Role
`lib/icStock.js`	All quantity-based IC stock work: `shipChallanStock` (~62), `restoreReturnedStock` (~166), `receiveChallanStock` (~244), `receivedInStock` (~326), `withdrawReceivedStock` (~344), `IcStockError` (422).
`lib/icReceive.js`	`receiveChallan` (~69, idempotent, guarded receivedAt stamp), `writeReceiptRegister` (~31), `undoReceive` (~102, not used by any route).
`lib/icRouting.js`	`allowedDestinations` (every other business), `checkRoute` (not to itself).
`lib/einvoice/provider.js`	`einvoiceMode()` (EINVOICE_MODE: mock default / sandbox / live), mock IRN generator, NIC client (`nicSession` , `nicCall`), `generateIrn` , `cancelIrn` , `CANCEL_REASONS` . The NIC client is **not yet tested against the sandbox**.
`lib/einvoice/payload.js`	`loadParties` , `buildPayload` (NIC schema 1.1) + `problems` list (GSTIN, PIN, address, HSN, doc number, same GSTIN).
`lib/docnumber.js`	`nextDocNumber` - IC invoice, APR record, Stock Adjustment numbers (not the DC number).
`lib/barcodeValue.js`	`sameBarcode` - matches POS lines to units.
`lib/refLabels.js` , `lib/listSort.js`	Display labels; header sorting for the DC list.
`components/NumberedPager.jsx`	`NumberedPager` , `SortHeader` , `useClientTable` (client-side sort/page for the custom IC pages).

14. Configuration

- **Doc Setup**: create document types **"Inter Company Sales Invoice"**, **"Auto Purchase Received"** and **"Stock Adjustment"** with a prefix. Without a prefix the invoice number is a bare padded number (e.g. 0001), which the NIC e-invoice rules reject (cannot start with 0).
- **DC numbers** are not from Doc Setup: `DC/<yy-yy>/<branch initials>/<seq>` from the Counter collection.
- **E-Invoice** (`.env`): `EINVOICE_MODE` (mock | sandbox | live), `EINVOICE_BASE_URL`, `EINVOICE_CLIENT_ID`, `EINVOICE_CLIENT_SECRET`, `EINVOICE_USERNAME`, `EINVOICE_PASSWORD`, `EINVOICE_GSTIN`, `EINVOICE_PUBLIC_KEY`. Mock produces a fake IRN and QR (printed with a MOCK banner). Never commit real credentials.
- **Business / Location** masters must have correct **GSTIN** (state code decides IGST vs CGST/SGST), address and PIN for e-invoice.
- **Database**: `.env` MONGODB_URI; restart the dev server after changing it (the connection is cached).

15. Recent changes (October 2026)

Date	Change
09-10	E-Invoice (mock mode) on the Sales Invoice: generate / cancel / print with IRN and QR.
13-10	Sales Invoice receiver filter (company → location auto-select) and Selected Items preview.
14-10	Auto Purchases Receiveds screen (pending with product group / HSN, received list, view).
17-10	Roles & Permissions on Return Approval and the IC lookups.
19-10	Numbered pager + sorting on the DC list; client-side sort/pager (`useClientTable`) on the four custom pages.
21-10	Close button on Delivery Challan Add/Edit.

16. How to run, test and debug

1. `npm install`, set `.env` (MONGODB_URI, AUTH_SECRET, EINVOICE_MODE=mock), `npm run dev`.
2. **Sender** (top bar = branch A): Consignment → Add, choose destination branch B, scan IN_STOCK barcodes, save, print.
3. **Receiver** (top bar = branch B): Receive Delivery Challan → Approve. Check new `barcodeLabel` rows under B with `icChallanId`.
4. Branch B: sell one of those barcodes at POS.
5. Branch A: Sales Invoice → the sold unit appears under Not Billed → tick → Save → E-Invoice → Generate (mock).
6. Branch B: Auto Purchases Receiveds → the invoice appears → choose group/HSN → Save.
7. Return: branch B Receive DC → View → Return Qty → Request; branch A Return Approval → Approve; check stock moved back.

Where to look first

- Scan refused on the challan → `app/api/ic-delivery-challan/item-lookup/route.js` and the unit's status/business/location.
- Stock not arriving at the receiver → `lib/icReceive.js` / `lib/icStock.js receiveChallanStock`; check the challan's `items[].stockMoves`.
- Sold unit missing from Not Billed → the receiver row's `icChallanId` and `sales[]` (`icBilledId` must be null).
- E-invoice errors → GET `.../einvoice` shows the payload and `problems`; `lib/einvoice/*`; Doc Setup prefix.
- 403 → Roles & Permissions ticks for the IC screens.
- Every recently changed file is listed by date in `change.md`.